

^{void} type_resultat nom_fonction ([liste des param formels]) ;

{ [déclaration locale]

instruction(s)

[return resultat;]

}

} sous-programme
appelé

Dans un programme appelant

1°/ Sans retour (void)

nom_fonction ([liste des param effectifs]) ;

2°/ Avec résultat & retour.

a/ type_resultat nom_variable;

nom_variable = nom_fonction ([liste des param effectifs]) ;

b/ printf(" le résultat est %? ", nom_fonction ([liste param eff])) ;

↑
selon le type_resultat

c/ l'expression = X + 5 * nom_fonction ([liste des param effectifs]) ;

Exemple

Exécution du programme

main

fonctionnel
dans notre exemple

Appel
des fonctions

Allocation
dynamique

Appelant

Appelé

Entrez un nombre positif: 5 ↵
 le factoriel de 5 est 120

nb	5
fact	120

N	5
f	120
i	120
j	120

int nl = nb

void echange(int* x, int* y)

{ int temp;

temp = *x;

*x = *y;

*y = temp;

printf("Dans echange a = %d et b = %d\n", *x, *y);

main()

{ int a, b;

printf("Entrez deux nombres:");

scanf("%d %d", &a, &b);

printf("Avant echange a = %d et b = %d\n", a, b);

echange(&a, &b);

printf("Après echange a = %d et b = %d\n", a, b);

{

Exécution:

Entre deux nombres: 3 7 ↘

Avant échange $a = 3$ et $b = 7$

Dans échange $a = 7$ et $b = 3$

Après échange $a = 3$ et $b = 7$

init $x = a$

init $y = b$

a	3	00FA
b	7	01B9
x	3 7	
y	7 3	
temp	3	

Entre deux nombres: 4 8 ↘

Avant échange $a = 4$ et $b = 8$

Dans échange $a = 8$ et $b = 4$

Après échange $a = 8$ et $b = 4$

init $x = a$

init $y = b$

a	4 8	00FA
b	8 4	01B9
x	00FA	
y	01B9	
temp	4	

init $g_x = a$

init $p = 5$

init $g_n = p$

$p = 15$

I	15 5 16	n
---	---------	---

Exécution de échange

Enter two numbers: 10 13

Avant échange $a = 10$ et $b = 13$

Après échange $a \leftrightarrow 13$ et $b \leftrightarrow 10$

$$\text{int } x, y = 6$$

a ~~13~~ 13 ~~x~~
 b ~~13~~ 10 ~~y~~

 hang ~~AD~~

sf